

Agenda

- Association
- Inheritance
- Types of inheritance
- Method Overriding
- Upcasting
- Downcasting
- Object class
- Methods of object class
 - `toString();`
 - `equals();`

Association

- If "has-a" relationship exist between the types, then use association.
- To implement association, we should declare instance/collection of inner class as a field inside another class.
- There are two types of associations
 - 1. Composition
 - 2. Aggregation

Composition

- Represents part-of relation i.e. tight coupling between the objects.
- The inner object is essential part of outer object.
- Heart is part of Human.
- Engine is part of Car.
- Wall is part of Room.
- joining date is a part of employee

Aggregation

- Represents has-a relation i.e. loose coupling between the objects.
- The inner object can be added, removed, or replaced easily in outer object.
- Car has a Driver.
- Company has Employees.
- Room has a window
- Employee has a vehicle

Inheritance

- If "is-a"/"kind-of" relationship exist between the types, then use inheritance.
- Inheritance is process of generalization to specialization.
- All members of parent class are inherited to the child class.
- Parent class is also called as super class and child class is also called as sub-class.
- Example:

- Manager is a Employee
- Mango is a Fruit
- Rectangle is a Shape
- In Java, inheritance is done using extends keyword.
- Java doesn't support multiple implementation inheritance i.e. a class cannot be inherited from multiple super-classes.
- However Java does support multiple interface inheritance i.e. a class can be inherited from multiple super interfaces.

Types of Inheritance

- 1. Single

```
class A {  
  
}  
class B extends A{  
  
}
```

- 2. Multiple

```
class A {  
  
}  
class B {  
  
}  
class C extends A,B{ // Not Allowed  
  
}  
  
interface I1{  
  
}  
interface I2{  
  
}  
  
interface I3 extends I1,I2{ // Allowed  
  
}  
  
class D implements I1,I2{ // Allowed  
  
}
```

- 3. Hirerachical

```
class A {  
  
}  
class B extends A{  
  
}  
class C extends A{  
  
}
```

- 4. Multilevel

```
class A {  
  
}  
class B extends A{  
  
}  
class C extends B{  
  
}
```

- Hybrid inheritance: Any combination of above types

Method Overriding

- Redefining a super-class method in sub-class with exactly same signature is called as "Method overriding".
- Programmer should override a method in sub-class in one of the following scenarios
 - 1. Super-class has not provided method implementation at all (abstract method).
 - 2. Super-class has provided partial method implementation and sub-class needs additional code. Here sub-class implementation may call super-class method (using super keyword).
 - 3. Sub-class needs different implementation than that of super-class method implementation.

Upcasting

- Assigning sub-class reference to a super-class reference.
- Sub-class "is a" Super-class, so no explicit casting is required.
- Using such super-class reference, only super-class methods inherited into sub-class can be called. This is "Object slicing".
- Using such super-class reference, super-class methods overridden into sub-class can also be called.

Polymorphism

- poly = Many , morphism = Forms
- It has two types
 1. compile time
 - implemented using method overloading
 - Compiler can identify which method to be called at compile time depending on types of arguments. This is also referred as "Early binding".
 2. Runtime - implemented using method overriding - The method to be called is decided at runtime depending on type of object. This is also referred as "Late binding" or "Dyanmic method dispatch".

SUNBEAM